Cross Validation Testing and K-Fold Cross Validation

Robust Model Evaluation on the Pima Indians Diabetes Dataset

Jayaram Gawade

July 21, 2026

Outline

Why Model Evaluation Matters

- A machine learning model is only as trustworthy as the way we **evaluate** it.
- The usual approach: split data into a **training set** and a **test set**.
- Question: *What if the split itself was just lucky (or unlucky)?*
- If accuracy changes drastically with different splits, we cannot trust a single number.

Core Problem

A single train/test split gives only **one** estimate of performance — it doesn't tell us how **stable** that estimate is.

The Problem: A Single Train/Test Split

Experiment: Same model (Logistic Regression), same data, 10 different random 80/20 splits.

Split	Accuracy	Split	Accuracy
1	0.8117	6	0.8117
2	0.7727	7	0.7922
3	0.7662	8	0.7727
4	0.7403	9	0.7468
5	0.7987	10	0.7468

Observation

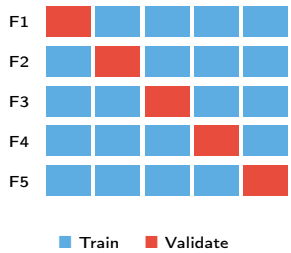
Accuracy ranges from **0.7403** to **0.8117** (a spread of **7.14%**) — purely due to which rows landed in the test set!

What is Cross-Validation?

- **Cross-Validation (CV)** is a resampling technique that uses *every* data point for both training and validation, across multiple rounds.
- Instead of one split, we perform several splits and **average** the results.
- This gives a performance estimate that is:
  - More **stable** (lower variance)
  - More **trustworthy** for comparing models
  - Accompanied by a measure of **uncertainty** (standard deviation)

K-Fold Cross-Validation: The Idea

1. Split the dataset into K equal-sized **folds**.
2. Train on $K-1$ folds, validate on the remaining fold.
3. Repeat K times, so every fold is used once for validation.
4. Average the K scores $\rightarrow$ final estimate.



$$\text{Final score} = \frac{1}{K}\sum_{i=1}^{K}\text{score}_i \qquad \pm\ \text{std}(\text{score}_i)$$

Stratified K-Fold

- Plain K-Fold splits data **randomly** into folds — this can create folds with very different class ratios.
- Our dataset is **imbalanced**: 500 non-diabetic (65.1%) vs. 268 diabetic (34.9%).
- **Stratified K-Fold** forces every fold to preserve the original class proportions.

Why it matters here

Without stratification, some folds could end up with too few diabetic cases, making validation accuracy misleading — especially risky in a **healthcare** context.

Dataset: Pima Indians Diabetes

Shape: 768 rows $\times$ 9 columns **Features (8):**

- Pregnancies, Glucose, BloodPressure
- SkinThickness, Insulin, BMI
- DiabetesPedigreeFunction, Age

Target: `Outcome`
(0 = No Diabetes, 1 = Diabetes)

Class	Count	%
No Diabetes	500	65.1%
Diabetes	268	34.9%

Data Cleaning: Hidden Missing Values

- No column has `NaN` values — but several medical columns contain **0s that are biologically impossible** (e.g. 0 Glucose).
- These 0s are treated as missing and replaced with the **column median**.

Column	Zero count	Median used
Glucose	5	117.0
BloodPressure	35	72.0
SkinThickness	227	29.0
Insulin	374	125.0
BMI	11	32.3

Step 1: Imports & Setup

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import (
    StratifiedKFold, cross_val_score
)
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
```

Step 2: Clean the Data

```
cols_to_fix = ['Glucose', 'BloodPressure',
               'SkinThickness', 'Insulin', 'BMI']

for col in cols_to_fix:
    median_val = df[col].replace(0, np.nan).median()
    df[col] = df[col].replace(0, median_val)

X = df.drop('Outcome', axis=1)
y = df['Outcome']
```

Step 3: The Naive Approach (Single Split)

```
from sklearn.model_selection import train_test_split

scores_list = []
model = LogisticRegression(max_iter=1000)

for seed in range(10):
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=seed
    )
    scaler = StandardScaler()
    X_train_s = scaler.fit_transform(X_train)
    X_test_s = scaler.transform(X_test)

    model.fit(X_train_s, y_train)
    scores_list.append(model.score(X_test_s, y_test))
```

Step 4: Stratified K-Fold Setup

```
skf = StratifiedKFold(n_splits=5, shuffle=True,
                      random_state=42)

# Pipeline ensures scaling happens INSIDE each fold
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('model', LogisticRegression(max_iter=1000))
])

scores = cross_val_score(
    pipeline, X, y, cv=skf, scoring='accuracy'
)

print(f"Mean Accuracy : {scores.mean():.4f}")
print(f"Std Deviation : {scores.std():.4f}")
```

Key detail

Scaling is fit *inside* the pipeline per fold, preventing data leakage from validation into training.

Step 5: Comparing Multiple Models

```
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000),
    'Decision Tree': DecisionTreeClassifier(max_depth=4,
                                            random_state=42),
    'KNN (k=5)': KNeighborsClassifier(n_neighbors=5)
}

results = {}
for name, model in models.items():
    pipe = Pipeline([('scaler', StandardScaler()),
                     ('model', model)])
    fold_scores = cross_val_score(pipe, X, y, cv=skf,
                                  scoring='accuracy')
    results[name] = fold_scores
```

Single Split vs. K-Fold CV

Method	Result	Reliability
Single 80/20 split	$0.7403 - 0.8117$	Unstable
5-Fold Stratified CV	0.7734 ± 0.0156	Stable

- K-Fold CV collapses a wide, unreliable range into a **single stable estimate with uncertainty**.
- 95% Confidence Interval: 0.7734 ± 0.0307.

Model Comparison (5-Fold Stratified CV)

Model	Mean Accuracy	Std Dev	Rank
Logistic Regression	**0.7734**	0.0156	1
Decision Tree	0.7291	0.0281	2
KNN (k=5)	0.7239	0.0297	3

- Logistic Regression is both the most **accurate** and the most **stable** (lowest std) of the three.
- Tree-based and distance-based models show higher fold-to-fold variance on this dataset.

Effect of K on Accuracy

K	Mean Accuracy	Std Dev
2	0.7643	0.0117
3	0.7682	0.0157
5	**0.7734**	0.0156
10	0.7668	0.0253
20	0.7694	0.0559

- Small K (e.g. 2–3): more bias, less variance, faster.
- Large K (e.g. 20): less bias, but variance across folds grows (small validation folds).
- $K = 5$ or $K = 10$ is the standard sweet spot.

Key Takeaways

1. A single split gave accuracy ranging **0.7403–0.8117** — unreliable, depends on the luck of the split.
2. K-Fold CV gives a stable **mean** $\pm$ **confidence interval** instead of one lucky/unlucky number.
3. **Stratified** K-Fold is essential for imbalanced data like this dataset.
4. In healthcare applications, **Recall** often matters more than Accuracy (avoid missed diagnoses).
5. $K = 5$ **or** $K = 10$ is the sweet spot for most datasets.

Thank You

Questions?